

Высшая школа информационных технологий и автоматизированных систем
(наименование высшей школы / филиала / института / колледжа)

По дисциплине DevTestOps

На тему Настройка системы vault

Груздев Николай Александрович
(Ф.И.О.)

09.03.01 Информатика и вычислительная техника

Группа: 151220

(отметка прописью)

(подпись руководителя)

Архангельск 2025

ЛИСТ ДЛЯ ЗАМЕЧАНИЙ

ЦЕЛИ РАБОТЫ

Получить практический опыт по настройке vault.

1 РЕШЕНИЕ ПОСТАВЛЕННОЙ ЗАДАЧИ

Установим необходимые утилиты для работы с HashiCorp Vault (рис. 1).

```
root@vm2:~# apt-get install wget chrony curl apt-transport-https
Чтение списков пакетов... Готово
Построение дерева зависимостей... Готово
Чтение информации о состоянии... Готово
Уже установлен пакет wget самой новой версии (1.21.3-1+deb12u1).
Уже установлен пакет curl самой новой версии (7.88.1-10+deb12u14).
```

Рисунок 1 – Установка системных утилит

Проверим корректность синхронизации времени на виртуальной машине (рис. 2).

```
root@vm2:~# timedatectl status
Local time: C6 2026-01-03 19:43:17 MSK
Universal time: C6 2026-01-03 16:43:17 UTC
RTC time: C6 2026-01-03 16:43:18
Time zone: Europe/Moscow (MSK, +0300)
System clock synchronized: yes
NTP service: active
```

Рисунок 2 – Проверка системного времени

Скачаем исполняемый файл Vault с Яндекс.Облака (рис. 3).

```
root@vm2:~# wget https://hashicorp-releases.yandexcloud.net/vault/1.21.1/vault_1.21.1_linux_amd64.zip
--2026-01-03 20:23:33-- https://hashicorp-releases.yandexcloud.net/vault/1.21.1/vault_1.21.1_linux_amd64.zip
Распознаётся hashicorp-releases.yandexcloud.net (hashicorp-releases.yandexcloud.net)... 51.250.76.140, 2a0d:d6c1:0:1a::2db
Подключение к hashicorp-releases.yandexcloud.net (hashicorp-releases.yandexcloud.net)|51.250.76.140|:443... соединение установлено.
```

Рисунок 3 – Загрузка Vault

Распакуем архив, переместим бинарник в /usr/local/bin/, настроим автодополнение команд и создадим рабочие директории (рис. 4).

```
root@vm2:~# unzip vault_1.21.1_linux_amd64.zip
Archive:  vault_1.21.1_linux_amd64.zip
replace LICENSE.txt? [y]es, [n]o, [A]ll, [N]one, [r]ename: y
  inflating: LICENSE.txt
  inflating: vault
root@vm2:~# mv vault /usr/local/bin/
root@vm2:~# vault --version
Vault v1.21.1 (2453aac2638a6ae243341b4e0657fd8aea1cbf18), built 2025-11-18T13:04:32Z
root@vm2:~# vault -autocomplete-install
```

Рисунок 4 – Подготовка среды для Vault

Создадим основной конфигурационный файл config.hcl (рис. 5).

```
ui = true

#mlock = true
#disable_mlock = true

storage "file" {
  path = "/opt/vault/data"
}
```

Рисунок 5 – Файл конфигурации config.hcl

Добавим переменную окружения с адресом сервиса, создадим пользователя vault и назначим ему права на рабочие директории (рис. 6).

```
root@vm2:~# touch /etc/vault.d/vault.env
root@vm2:~# echo "export VAULT_ADDR=http://127.0.0.1:8201" > /etc/vault.d/vault.
env
root@vm2:~# source /etc/vault.d/vault.env
root@vm2:~# useradd --system --home /etc/vault --shell /bin/false vault
root@vm2:~# chown -R vault:vault /etc/vault.d /opt/vault
```

Рисунок 6 – Настройка переменной окружения и пользователя vault

Создадим unit-файл systemd для управления сервисом (рис. 7).

```
[Unit]
Description="HashiCorp Vault - A tool for managing secrets"
Documentation=https://www.vaultproject.io/docs/
Requires=network-online.target
After=network-online.target
ConditionFileNotEmpty=/etc/vault.d/config.hcl

[Service]
User=vault
Group=vault
ProtectSystem=full
ProtectHome=read-only
PrivateTmp=yes
PrivateDevices=yes
SecureBits=keep-caps
AmbientCapabilities=CAP_IPC_LOCK
NoNewPrivileges=yes
ExecStart=/usr/local/bin/vault server -config=/etc/vault.d/config.hcl
ExecReload=/bin/kill --signal HUP
KillMode=process
```

Рисунок 7 – Файл сервиса vault.service

Активируем и запустим Vault (рис. 8).

```
root@vm2:~# systemctl status vault
● vault.service - "HashiCorp Vault - A tool for managing secrets"
   Loaded: loaded (/etc/systemd/system/vault.service; enabled; preset: enable>
   Active: active (running) since Sat 2026-01-03 20:59:40 MSK; 6s ago
```

Рисунок 8 – Запуск сервиса Vault

Веб-интерфейс Vault доступен и работает (рис. 9).

Let's set up the initial set of root keys that you will need in case of an emergency.

Key shares

The number of key shares to split the root key into

Key threshold

The number of key shares required to reconstruct the root key

☐ Encrypt output with PGP

☐ Encrypt root token with PGP

Initialize

Рисунок 9 – Веб-интерфейс Vault

Инициализируем Vault. В результате получены ключи для разблокировки и root-токен (рис. 10).

```
root@vm2:~# vault operator init
Unseal Key 1: jNabKeXsCNAoH5DyaSezD/s5mBcmhbyf++X22kQ7uA8z
Unseal Key 2: IZ2xAs+hvc4aR00N+H/emUpv+06LZZ0H0D9GYi1Gy6zs
Unseal Key 3: u8U3f9LTee+Bvk2rwZ9xmd1GEPfv0X8dbKIbgolpPlYR
Unseal Key 4: ryg3MyWbTW4qWx/q40j5TNa6yxpqauIHIAccF+bxH9Ui
Unseal Key 5: MtGibYhAmYEou/qgkHjbQ/G5uGA4YvsESdEz9DB7Bdfe

Initial Root Token: hvs.D7hRs1HJSIZhETPon6M85s9P
```

Рисунок 10 – Инициализация Vault

Разблокируем сервис, введя три из пяти ключей (рис. 11).

```
root@vm2:~# vault operator unseal
Unseal Key (will be hidden):
Key      Value
---      -
Seal Type  shamir
Initialized true
Sealed     false
Total Shares 5
Threshold  3
Version     1.21.1
Build Date  2025-11-18T13:04:32Z
Storage Type file
Cluster Name vault-cluster-3535a23e
Cluster ID  57c4a729-34e8-4bcc-d1ca-984ec8876065
HA Enabled  false
```

Рисунок 11 – Распечатывание (unseal) Vault

Авторизуемся с root-токеном для выполнения операций (рис. 12).

```

root@vm2:~# vault login
Token (will be hidden):
Success! You are now authenticated. The token information displayed below
is already stored in the token helper. You do NOT need to run "vault login"
again. Future Vault requests will automatically use this token.

Key          Value
---          -
token        hv5.D7hRs1HJSIZhETPon6M85s9P
token_accessor PJ8tuYgSlUyi0C4V5CKWNU6I
token_duration ∞
token_renewable false
token_policies ["root"]
identity_policies []
policies       ["root"]

```

Рисунок 12 – Авторизация под root

Активируем механизм хранения секретов по пути secret и создадим простой секрет (рис. 13).

```

root@vm2:~# vault secrets enable -path=secret/ kv
Success! Enabled the kv secrets engine at: secret/
root@vm2:~# vault kv put secret/hello foo=world
Success! Data written to: secret/hello
root@vm2:~# vault kv get secret/hello
=== Data ===
Key    Value
---    -
foo    world

```

Рисунок 13 – Создание секрета

Изменим секрет, добавив второе значение, прочитаем его и удалим (рис. 14).

```

root@vm2:~# vault kv put secret/hello foo=world excited=yes
Success! Data written to: secret/hello
root@vm2:~# vault kv list secret
Keys
----
hello
root@vm2:~# vault kv get secret/hello
===== Data =====
Key    Value
---    -
excited yes
foo    world
root@vm2:~# vault kv delete secret/hello
Success! Data deleted (if it existed) at: secret/hello

```

Рисунок 14 – Редактирование и удаление секрета

Включим версиюность секретов (версия 2). После двух изменений ключа foo сохраняются обе версии (рис. 15).

```

root@vm2:~# vault secrets enable -version=2 kv
Success! Enabled the kv secrets engine at: kv/
root@vm2:~# vault kv enable-versioning secret/
Success! Tuned the secrets engine at: secret/
root@vm2:~# vault kv put secret/hello foo=world
== Secret Path ==
secret/data/hello

===== Metadata =====
Key                Value
---              -
created_time       2026-01-03T18:40:43.129567703Z
custom_metadata    <nil>
deletion_time      n/a
destroyed          false
version            1
root@vm2:~# vault kv put secret/hello foo=world2
== Secret Path ==
secret/data/hello

===== Metadata =====
Key                Value
---              -
created_time       2026-01-03T18:40:53.114121971Z
custom_metadata    <nil>
deletion_time      n/a
destroyed          false
version            2

```

Рисунок 15 – Версионность секрета

Настроим динамическое создание учётных записей в MariaDB. Создадим пользователя vaultuser с правами управления пользователями и активируем соответствующий механизм секретов (рис. 16).

```

root@vm2:~# mysql -uroot -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 31
Server version: 10.11.14-MariaDB-0+deb12u2 Debian 12

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> CREATE USER 'vaultuser'@'localhost' IDENTIFIED BY 'vaultpass';

Query OK, 0 rows affected (0.010 sec)

MariaDB [(none)]> GRANT CREATE USER ON *.* TO 'vaultuser'@'localhost' WITH GRANT
OPTION;
Query OK, 0 rows affected (0.002 sec)

MariaDB [(none)]> exit;
Bye
root@vm2:~# vault secrets enable database
Success! Enabled the database secrets engine at: database/

```

Рисунок 16 – Настройка доступа к БД и активация механизма

Добавим конфигурацию подключения к БД и создадим роль для генерации временных учётных записей (рис. 17).


```

root@vm2:~# vault write database/config/test \
plugin_name=mysql-database-plugin \
connection_url="{{username}}:{{password}}@tcp(127.0.0.1:3306)/" \
allowed_roles="test-role" \
username="vaultuser" \
password="vaultpass"
Success! Data written to: database/config/test
root@vm2:~# vault write database/roles/test-role \
db_name=test \
creation_statements="CREATE USER '{{name}}'@'%' IDENTIFIED BY '{{password}}';" \
default_ttl="1h" \
max_ttl="24h"
Success! Data written to: database/roles/test-role

```

Рисунок 17 – Настройка подключения и роли БД

Сгенерируем временные логин и пароль через CLI (рис. 18).

```

root@vm2:~# vault read database/creds/test-role
Key          Value
---          -
lease_id     database/creds/test-role/ywoZi9GjfsabmNjLodN0mEb4
lease_duration 1h
lease_renewable true
password     94-gWwt22VWatBGsxy2q
username     v-root-test-role-pVjvurpgt0TCnW8

```

Рисунок 18 – Создание временного пользователя через CLI

Аналогично создадим пользователя через API с помощью curl (рис. 19).

```

root@vm2:~# curl --header "X-Vault-Token: hvs.D7hRs1HJSIZhETPon6M85s9P" http://127.0.0.1:8201/v1/database/creds/test-role
{"request_id": "e5fe9d4e-c463-f9d4-9568-e8f02264bb7a", "lease_id": "database/creds/test-role/e7vnFBkBEEPHptuhcErK2iXR", "renewable": true, "lease_duration": 3600, "data": {"password": "hKv89jTct-uFgT7C-4ZT", "username": "v-root-test-role-4c5IYl9uRiG8mNY"}, "wrap_info": null, "warnings": null, "auth": null, "mount_type": "database"}

```

Рисунок 19 – Генерация пользователя через API

Проверим список пользователей в MariaDB — временные учётные записи созданы (рис. 20).

```

MariaDB [(none)]> select user, host from mysql.user;
+-----+-----+
| User          | Host          |
+-----+-----+
| mariadb.sys   | localhost    |
| mysql         | localhost    |
| root          | localhost    |
| v-root-test-role-4c5IYl9uRiG8mNY | localhost    |
| v-root-test-role-pVjvurpgt0TCnW8 | localhost    |
| vaultuser     | localhost    |
+-----+-----+
6 rows in set (0.001 sec)

```

Рисунок 20 – Пользователи в БД MariaDB

Успешно авторизуемся в консоли MariaDB под временным пользователем (рис. 21).

```
root@vm2:~# mysql -u'v-root-test-role-pVjvurpgt0TCnW8' -p'94-gWwt22VWatBGsxy2q'  
Welcome to the MariaDB monitor.  Commands end with ; or \g.  
Your MariaDB connection id is 41  
Server version: 10.11.14-MariaDB-0+deb12u2 Debian 12  
  
Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.  
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
```

Рисунок 21 – Авторизация под временным пользователем

Создадим политику доступа и токен с её использованием (рис. 22).

```
root@vm2:~# vault policy write my-policy - << EOF  
path "secret/data/foo/*" {  
  capabilities = ["read", "create", "update"]  
}  
  
path "secret/data/hello/*" {  
  capabilities = ["read", "update"]  
}  
  
path "secret/data/*" {  
  capabilities = ["read"]  
}  
EOF  
Success! Uploaded policy: my-policy  
root@vm2:~# vault token create -policy=my-policy  
Key          Value  
---          -  
token        hvs.CAESIFiXFtvMaOicckTFLria  
2cy5aenF6d3FEaVRQZDM4ZkZUUjlWY2l0eFg  
token_accessor DqE16zbwuzRmRbIapMH6wUPM  
token_duration 768h  
token_renewable true  
token_policies ["default" "my-policy"]  
identity_policies []  
policies       ["default" "my-policy"]
```

Рисунок 22 – Создание политики и токена

Авторизуемся с новым токеном (рис. 23).

```
root@vm2:~# vault login  
Token (will be hidden):  
Success! You are now authenticated. The token information displayed below  
is already stored in the token helper. You do NOT need to run "vault login"  
again. Future Vault requests will automatically use this token.  
  
Key          Value  
---          -  
token        hvs.CAESIFiXFtvMaOicckTFLriaJ9iZsh2QRipmF_dPvZ55yTPTGh  
2cy5aenF6d3FEaVRQZDM4ZkZUUjlWY2l0eFg  
token_accessor DqE16zbwuzRmRbIapMH6wUPM  
token_duration 767h58m50s  
token_renewable true  
token_policies ["default" "my-policy"]  
identity_policies []  
policies       ["default" "my-policy"]
```

Рисунок 23 – Авторизация под токеном с политикой

Проверим работу политики. Создание секрета в secret/foo разрешено, а в secret/hello — отклонено (рис. 24).

```
root@vm2:~# vault kv put secret/foo/new foo=world
=== Secret Path ===
secret/data/foo/new

===== Metadata =====
Key          Value
---          -
created_time  2026-01-06T23:28:01.36250467Z
custom_metadata  <nil>
deletion_time  n/a
destroyed     false
version       1
root@vm2:~# vault kv put secret/hello/new foo=world
Error writing data to secret/data/hello/new: Error making API request.

URL: PUT http://127.0.0.1:8201/v1/secret/data/hello/new
Code: 403. Errors:

* 1 error occurred:
* permission denied
```

Рисунок 24 – Тестирование политики доступа

Вернёмся под root и создадим секрет в защищённой директории secret/hello (рис. 25).

```
root@vm2:~# vault login
Token (will be hidden):
Success! You are now authenticated. The token information displayed below
is already stored in the token helper. You do NOT need to run "vault login"
again. Future Vault requests will automatically use this token.

Key          Value
---          -
token        hvs.D7hRs1HJSIZhETPon6M85s9P
token_accessor PJ8tuYgSlUyi0C4V5CKWNU6I
token_duration ∞
token_renewable false
token_policies ["root"]
identity_policies []
policies       ["root"]
root@vm2:~# vault kv put secret/hello/new foo=world
=== Secret Path ===
secret/data/hello/new

===== Metadata =====
Key          Value
---          -
created_time  2026-01-06T23:29:45.757020178Z
custom_metadata  <nil>
deletion_time  n/a
destroyed     false
version       1
```

Рисунок 25 – Создание секрета под root

Под токеном с политикой обновим существующий секрет — операция разрешена (рис. 26).

```

root@vm2:~# vault login
Token (will be hidden):
Success! You are now authenticated. The token information displayed below
is already stored in the token helper. You do NOT need to run "vault login"
again. Future Vault requests will automatically use this token.

Key          Value
---          -
token        hvs.CAESIFiXFtvMaOicckTFLriaJ9iZsh2QRipmF_dPvZ55yTPTGh4KHGh
2cy5aenF6d3FEaVRQZDM4ZkZUUjlWY2l0eFg
token_accessor DqE16zbwuzRmRbiapMH6wUPM
token_duration 767h54m6s
token_renewable true
token_policies ["default" "my-policy"]
identity_policies []
policies       ["default" "my-policy"]
root@vm2:~# vault kv put secret/hello/new foo=world2
==== Secret Path ====
secret/data/hello/new

===== Metadata =====
Key          Value
---          -
created_time 2026-01-06T23:31:33.1045181Z
custom_metadata <nil>
deletion_time n/a
destroyed     false
version       2

```

Рисунок 26 – Обновление секрета в рамках политики

ВЫВОД

В ходе работы были получены практические навыки по развёртыванию и настройке HashiCorp Vault, включая управление секретами, настройку динамических учётных записей БД и работу с политиками доступа.